SOFTWARE REQUIREMENTS & PROBLEM STATEMENT

Automated Software Verification & Testing Platform

A systematic platform for requirement-driven test generation, execution, defect tracking, and verification traceability

Document Version 1.0 | Problem Statement Document

1. Introduction

1.1 Purpose

The purpose of this document is to define the problem statement, scope, and functional requirements for a software platform that helps development teams systematically verify and test their applications. Manual and ad-hoc testing practices frequently lead to incomplete test coverage, undocumented defects, and a lack of traceability between requirements and the tests that validate them. This document describes a platform that addresses these gaps by combining structured test case management, automated test generation using formal testing techniques (Equivalence Partitioning and Boundary Value Analysis), multi-layer test execution (unit, API, integration, regression, and UI), defect management, and end-to-end requirement-to-defect traceability, all surfaced through a consolidated testing dashboard.

This document is intended to serve as the foundation for design and implementation, and to make the verification methodology behind the platform explicit enough to be defended in an academic or evaluative setting.

1.2 Document Conventions

The following conventions are used throughout this document:

- **Bold text** is used to highlight key terms, module names, and section headers.
- `Monospace text` denotes code, function signatures, API parameters, and system identifiers (e.g., `withdraw(amount)`).
- Requirement and test artifacts follow the naming convention `TC-XXX` for test cases and `BUG-XXX` for defects, where `XXX` is a zero-padded sequence number.
- Priority levels follow the standard scale P0 (Critical) through P3 (Low); severity levels follow Critical, High, Medium, and Low.
- Flowcharts represent unidirectional data or process flow from top to bottom unless otherwise labeled.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

- **Developers and QA Engineers** — to understand the test generation logic, supported test types, and integration points with existing codebases.
- **Project Evaluators / Academic Reviewers** — to assess the verification methodology, requirement traceability model, and the rigor of the testing approach (Section 6 in particular).
- **Project Team Members** — to use as a shared reference for scope, features, and the recommended technology stack during implementation.
- **Future Maintainers** — to understand the system's architecture and extend it, particularly the optional AI-assisted test generation component.

Readers primarily interested in system behavior should focus on Sections 2 and 3. Readers evaluating the academic or methodological strength of the project should focus on Section 3.6 (Verification Traceability) and Section 3.2 (Automated Test Generation), which demonstrate the application of formal software testing techniques.

1.4 Project Scope

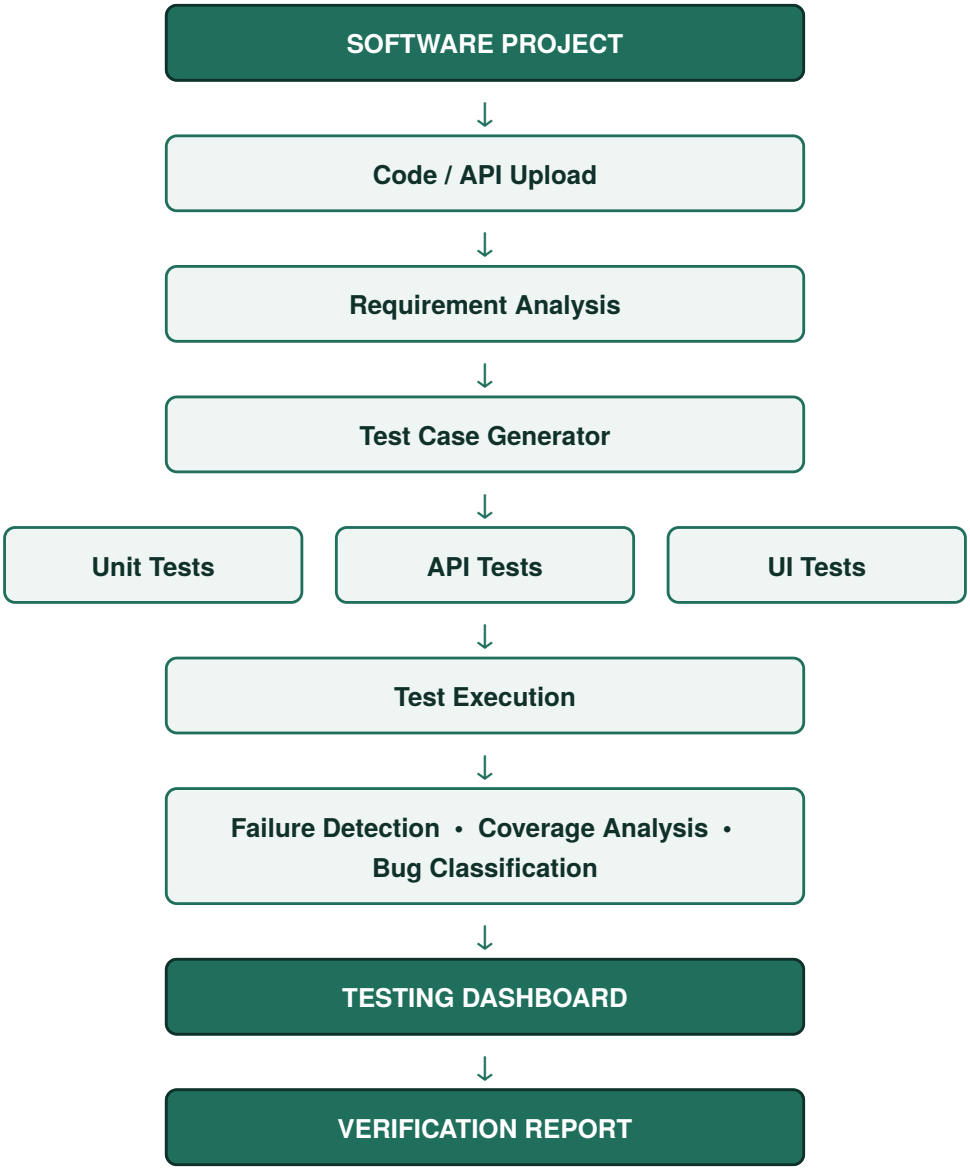
The platform allows a developer to upload a software project's code or API definition and receive a structured, traceable verification workflow in return. In scope for the initial version of the platform:

- Manual and automated creation of test cases, organized into test suites and mapped to requirements.
- Automated generation of test cases using Equivalence Partitioning and Boundary Value Analysis, based on function signatures or requirement text.
- Execution of unit, API, integration, regression, and UI tests through integration with established testing frameworks.
- Automatic failure detection, code coverage analysis, and defect classification by severity and module.
- A consolidated testing dashboard summarizing pass/fail metrics, coverage, and defect distribution.
- End-to-end requirement-to-defect traceability for verification reporting.

Out of scope for the initial version: performance/load testing, security penetration testing, and production deployment monitoring. These are noted as candidates for future extension but are not part of the core workflow described in this document.

2. Core Workflow

The platform follows a linear pipeline from code ingestion through to a verification report, branching across three test types during execution. The end-to-end flow is illustrated below.



3. Features

3.1 Test Case Management

- Create, edit, and delete test cases
- Assign priority (P0–P3)
- Track status: Pass / Fail / Blocked
- Organize test cases into test suites
- Map each test case to its originating requirement

3.2 Automated Test Generation

Given a function signature such as `withdraw(amount)`, the platform automatically generates a set of test cases using two established black-box testing techniques:

Input	Technique	Expected Outcome
`amount = 100`	Equivalence Partitioning (valid class)	Pass
`amount = 0`	Boundary Value Analysis (lower boundary)	Fail / Reject
`amount = -100`	Equivalence Partitioning (invalid class)	Reject
`amount > balance`	Equivalence Partitioning (invalid class)	Reject
`amount = maximum allowed`	Boundary Value Analysis (upper boundary)	Pass

This is the core academic contribution of the platform: it demonstrates a working, automated application of Equivalence Partitioning and Boundary Value Analysis rather than relying on manually authored test data.

3.3 Automated Testing

The platform supports execution across the following test types:

- Unit testing
- API testing
- Integration testing
- Regression testing
- UI testing

3.4 Defect Management

When a test fails, the platform automatically opens a defect record linked back to its test case:

BUG #104

Severity: HIGH
Priority: P1
Test Case: TC-023
Expected: ₹500 withdrawal
Actual: ₹1000 withdrawal
Status: OPEN

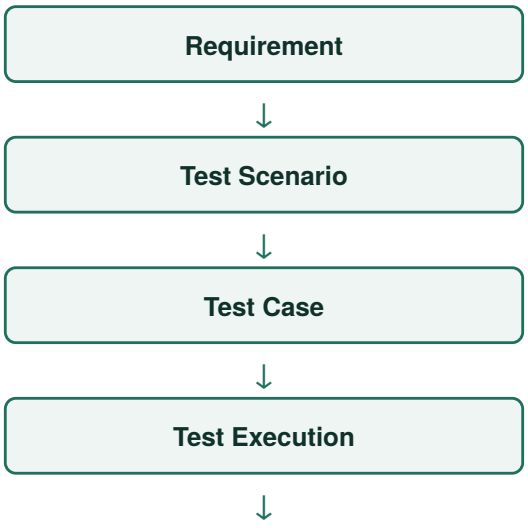
3.5 Testing Dashboard

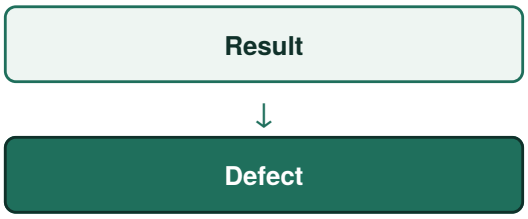
The dashboard consolidates the following metrics in real time:

- Total test cases
- Passed / Failed / Blocked counts
- Pass percentage
- Code coverage
- Defects by severity
- Defects by module
- Regression results

3.6 Verification Traceability

Every requirement is traceable through to its resulting defects, forming a complete verification chain. This traceability model is the component that makes the project academically defensible, since it demonstrates formal requirements verification rather than isolated test execution.




Result
Defect

4. Recommended Technology Stack

The stack below is selected based on team familiarity and the platform's requirements for real-time dashboards, multi-type test execution, and an optional AI-assisted generation component.

Frontend

- React
- Tailwind CSS
- Recharts

Backend

- Python
- Flask / FastAPI

Database

- PostgreSQL / MySQL

Testing

- PyTest
- Selenium or Playwright
- Postman / Newman or REST API testing
- Coverage.py

Optional AI Component

- LLM API for generating test cases directly from natural-language requirements